

Technical Report
Shadow Bank
Bhavpreet Singh
1291491
CSCI-300
Database Management
Fall 2023
Robert Doxey
12/18/23

<https://github.com/fangedShadow/shadowBank>

Introduction	2
Technologies and tools	2
Users and Features	2

Database Design	3
Principle	3
Entity Relationship Diagram	5
Implementation	5
Conclusion.....	15
Limitations:	15
Challenges:.....	15
Future Upgrades:	15

Introduction

Welcome to the world of Shadow Bank, where technology meets finance! This report sheds light on a Java program that collaborates with MySQL to power our banking system. This application is the backbone of our operations, ensuring smooth account management and secure transactions. As we journey through the report, we'll uncover how Java and MySQL work hand in hand to make Shadow Bank.

Technologies and tools

Programming Language: - Java

The core of the Shadow Bank system is built upon the Java programming language, renowned for its platform independence and versatility. Java's object-oriented approach provides a solid foundation for developing the various components and functionalities required for operations.

Database: - MySQL

Managing the data backend of Shadow Bank is the MySQL database. MySQL, a popular open-source relational database management system, excels in handling large datasets efficiently. It ensures the secure storage and retrieval of information.

UI Designer: - Window Builder

The user interface (UI) of the Shadow Bank application is crafted using Window Builder, a powerful and user-friendly tool for designing graphical user interfaces in Java. With its drag-and-drop functionality, Window Builder simplifies the UI design process, allowing developers to create visually appealing and intuitive interfaces that enhance the overall user experience.

Users and Features

1.Customer:

- Login and Register: Customers can create an account by registering and subsequently log in securely to access their banking services.
- Account Management: Customers can manage one or multiple bank accounts associated with their profile, providing flexibility in financial operations.
- Profile Modification: Users can change their profile details, ensuring that their information is up-to-date and accurate.
- Financial Transactions: Customers can perform financial transactions such as depositing and withdrawing money, ensuring control over their account balances.
- Online Appointment Booking: Customers have the convenience of booking online appointments for personalized assistance and can view messages sent by bank employees for important updates.

2.Employee:

- Customer and Account Management: Employees have access to a list of all customers and their associated bank accounts, facilitating efficient customer service.
- Bank Account Operations: Employees can delete bank accounts when necessary and can accept or decline new bank account requests, maintaining control over the customer base.
- Appointment Handling: Employees can view and manage appointments booked by customers, allowing for organized and timely interactions.
- Customer Communication: Employees can send messages to customers and respond to their inquiries, ensuring effective communication.

3.Administrator:

- Employee Account Creation: Admins have the authority to create employee accounts, ensuring that the workforce is appropriately managed.
- Account Deletion: Admins can delete customer accounts, employee accounts, and bank accounts, providing ultimate control over the system's user base.
- Profile Modification: Admins can change the profile account details of both customers and employees, maintaining accuracy and compliance.

Database Design

Principle

The Database created for the bank Application had three Users: admin, Employee and Customer. The admin account has an ID (which is automatically given to the account when a new account is created), Username, Password, and name.

```
create table shadowbank.adminAcc(
adminID int AUTO_INCREMENT PRIMARY KEY,
adminUserName varchar(255) UNIQUE,
adminPassword varchar(255),
adminName varchar(255)
);
```

The Customer account has an ID (which is automatically given to the account when a new account is created), Username, Password, Name, Address.

```
create table shadowbank.custAcc(
custID int AUTO_INCREMENT PRIMARY KEY,
custUserName varchar(255) UNIQUE,
custPassword varchar(255),
custName varchar(255),
custAddress varchar(255)
);
```

A customer can request the creation of a bank account. The request has an ID (which is automatically given to the account when a new account is created), Customer ID (which is foreign key and is from the customer which requested the account), amount, approval (is null initially but is changed by the Employee depending on the acceptance).

```
create table shadowbank.BArequest(
requestID int AUTO_INCREMENT PRIMARY KEY,
custID int,
amount int,
aproval boolean,
FOREIGN KEY(custID) REFERENCES custAcc(custID)
);
```

The employee account has an ID (which is automatically given to the account when a new account is created), Username, Password, Name.

```
create table shadowbank.empAcc(
empID int AUTO_INCREMENT PRIMARY KEY,
empUserName varchar(255) UNIQUE,
empPassword varchar(255),
empName varchar(255)
);
```

An Employee can message a customer. The Message contains a message content, customer ID (to which it is going), Employee ID (the employee which is sending the message)

```
create table shadowbank.comMsg(
custID int,
empID int,
message text,
FOREIGN KEY(custID) REFERENCES custAcc(custID),
FOREIGN KEY(empID) REFERENCES empAcc(empID)
);
```

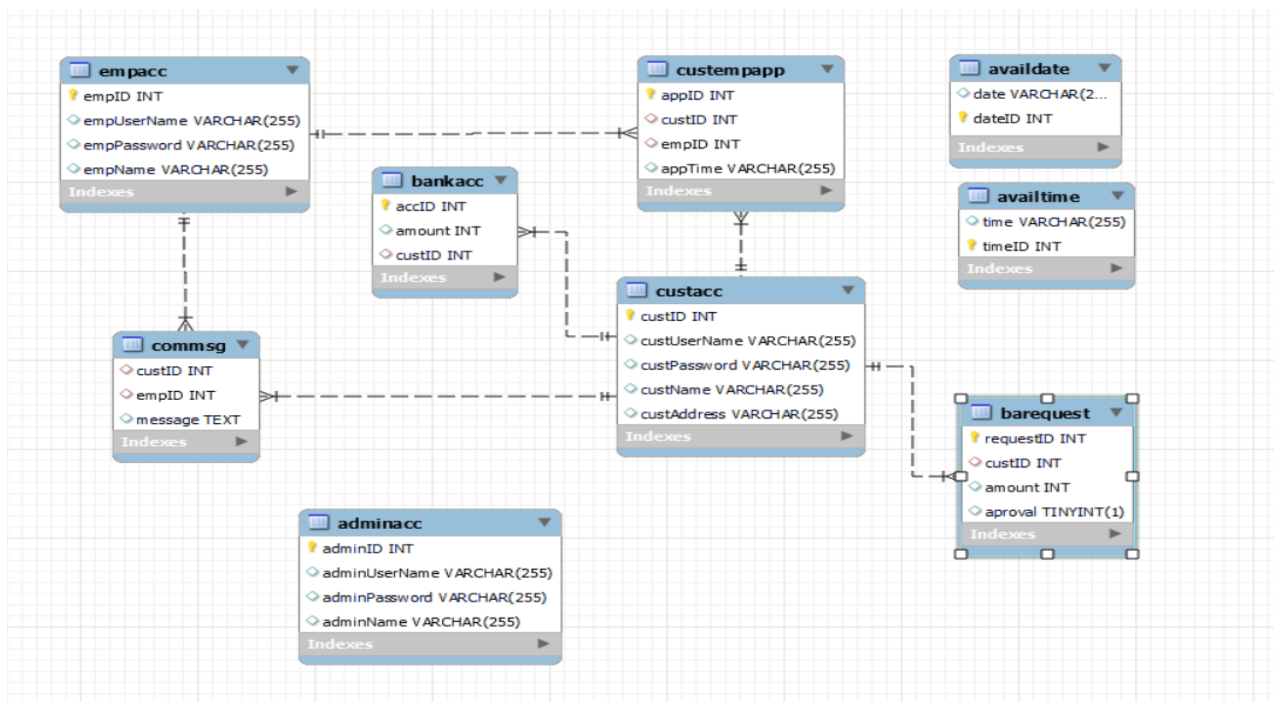
A customer can book an appointment with the employee. The customer can choose from the available date (has the dates and ID) and time (has the dates and ID), and employee. Then it is stores in the custEmpApp which has the customer ID (the customer that booked), employee ID (with whom the appointment is), time (stores the date and time of the appointment)

```
create table shadowbank.custEmpApp(
  appID int AUTO_INCREMENT PRIMARY KEY,
  custID int,
  empID int,
  appTime varchar(255),
  FOREIGN KEY(custID) REFERENCES custAcc(custID),
  FOREIGN KEY(empID) REFERENCES empAcc(empID)
);

create table shadowbank.availdate(
  date varchar(255),
  dateID int primary key AUTO_INCREMENT
);

create table shadowbank.availtime(
  time varchar(255),
  timeID int primary key AUTO_INCREMENT
);
```

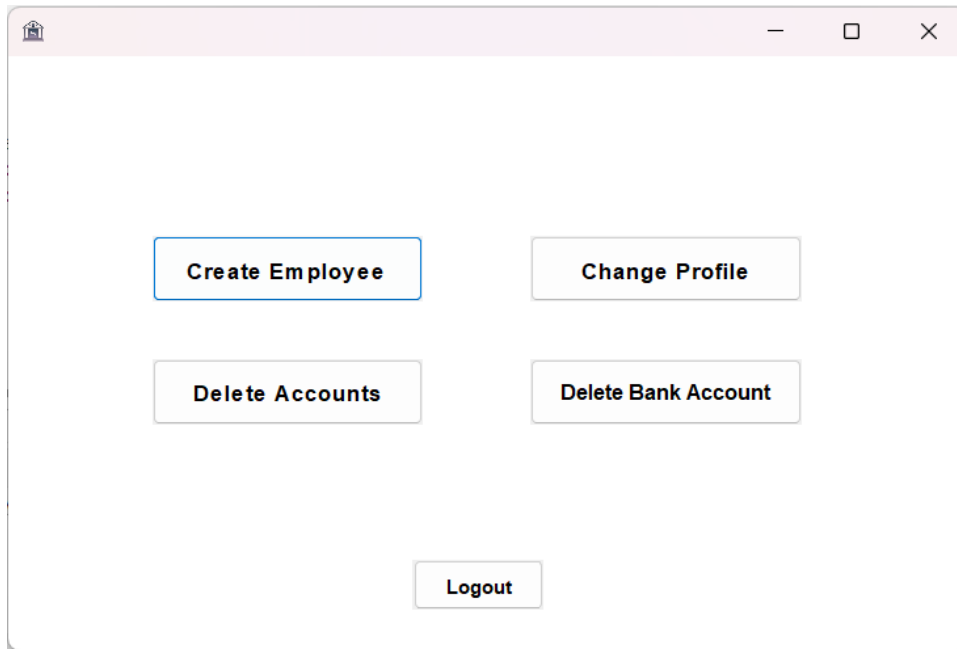
Entity Relationship Diagram



Implementation

In this section we will explain different important snippets of code that were used. All the snippets would be the methods which were interacting with the UI.

The basic button presses



```
 JButton empBtn = new JButton("Create Employee");
 empBtn.setFont(new Font("SansSerif", Font.BOLD, 14));
 empBtn.addActionListener(new ActionListener() {
     public void actionPerformed(ActionEvent e) {
         goToACE();
     }
     private void goToACE() {
         // TODO Auto-generated method stub
         frame.dispose();
         adminCreateEmp ACE = new adminCreateEmp();
         ACE.initialize();
         ACE.frame.setVisible(true);
     }
 });
```

when the button (empBtn) which is create employee is clicked, the code disposes of the current frame, creates a new instance of the adminCreateEmp class, initializes its components, and makes the new frame visible, presumably switching to a new interface for creating employee accounts. We used a similar method across other classes to switch between windows.

Changing profile information

This class can change the employee and customer account details.

Customer ID

3

Username

Full Name

Address

Password

Change

Employee ID

1

Username

Full Name

Password

Change

```

public void populateCustCB() {
    try {
        if (Database.connection == null) {
            Database.connect();
        }

        Connection connection = Database.connection;
        String query = "SELECT * FROM shadowbank.custacc";
        custCBmodel = new DefaultComboBoxModel<String>();

        try (PreparedStatement stm = connection.prepareStatement(query)) {
            ResultSet result = stm.executeQuery();

            while (result.next()) {
                String custID = result.getString("custID");
                custCBmodel.addElement(custID);
            }

            custCB.setModel(custCBmodel);
        } catch (Exception e) {
            e.printStackTrace(); // Handle the exception according to your needs
        }
    } catch (Exception e) {System.out.println(e);
    }
}

```

Java method, `populateCustCB()`, is designed to populate a `JComboBox` (`custCB`) with customer IDs retrieved from a database table named `custacc` in the "shadowbank" schema. The method establishes a database connection, executes a `SELECT` query to retrieve all customer IDs, and dynamically adds them to a `DefaultComboBoxModel` (`custCBmodel`). The `JComboBox` is then set to display the populated model, allowing users to select from the available customer IDs. The method includes exception handling to manage potential errors during the database interaction,

ensuring robustness in handling unexpected situations. We have the same method in other class to get data in a combobox.

```
private boolean updateEmpProfile(int empID ,String username, char[] password, String newName) {  
    try {  
        if (Database.connection == null) {  
            Database.connect();  
        }  
        Connection connection = Database.connection;  
        String nPassword = new String(password);  
        // Use a PreparedStatement to prevent SQL injection  
        String query = "UPDATE empacc SET empUserName = ?, empPassword = ?, empName = ? WHERE empID = ?";  
        PreparedStatement stm = connection.prepareStatement(query);  
        stm.setString(1, username);  
        stm.setString(2, nPassword);  
        stm.setString(3, newName);  
        stm.setInt(4, empID);  
  
        // Execute the update  
        int rowsAffected = stm.executeUpdate();  
  
        // Close the PreparedStatement (optional)  
        stm.close();  
  
        // Return true if at least one row was updated  
        return rowsAffected > 0;  
    } catch (Exception ex) {  
        ex.printStackTrace();  
        return false;  
    }  
}
```

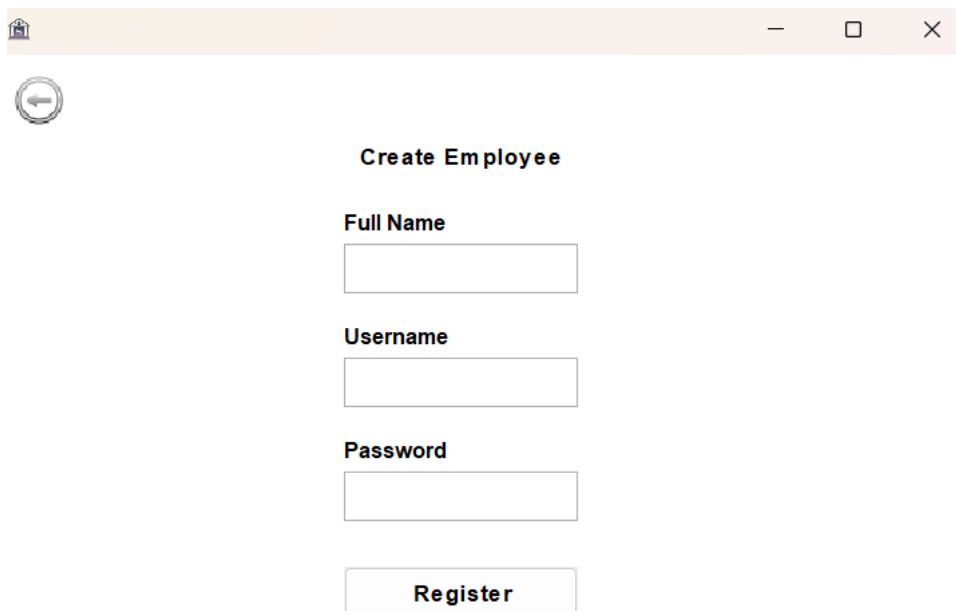
Java method, `updateEmpProfile`, is responsible for updating the profile of an employee in the "empacc" table of a database. It takes as parameters the employee ID (`empID`), the new username (`username`), password (`password`), and the new name (`newName`). The method establishes a database connection, utilizes a `PreparedStatement` to prevent SQL injection, and constructs an SQL query for updating the specified employee's profile. The `PreparedStatement` is set with the provided parameters, and the update is executed. The method returns a boolean value indicating whether the update was successful, with `true` indicating that at least one row was affected, and `false` otherwise. We have a similar method in different class to update the info.

```
JButton custChangeBtn = new JButton("Change");  
custChangeBtn.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        int selectedCustID = Integer.parseInt((String) custCB.getSelectedItem());  
        String username = custUN.getText();  
        char[] password = custPass.getPassword();  
        String newName = custName.getText();  
        String newAddress = custAdd.getText();  
  
        // Check if both name and address are not empty  
        if (!newName.isEmpty() && !newAddress.isEmpty()) {  
            // Update the user's profile in the database  
            if (updateCustProfile(selectedCustID, username, password, newName, newAddress)) {  
                System.out.println("Customer Account Updated!!");  
                // Optionally, you can clear the text fields after a successful update  
                custUN.setText("");  
                custPass.setText("");  
                custName.setText("");  
                custAdd.setText("");  
            } else {  
                System.out.println("Failed to update customer account");  
            }  
        } else {  
            System.out.println("Name and address cannot be empty");  
        }  
    }  
});
```

This Java code involves a JButton (custChangeBtn) with an ActionListener to handle changes to a customer's account information in a graphical user interface. When the button is clicked, the ActionListener retrieves the selected customer ID from a JComboBox (custCB) and gathers the new username, password, name, and address from corresponding text fields (custUN, custPass, custName, custAdd). It then validates that both the new name and address are not empty. If valid, it calls the updateCustProfile method to update the customer's profile in the database. If the update is successful, a message is printed, and the text fields are optionally cleared. If the update fails or if the name and address are empty, appropriate messages are printed to the console. We do the same thing when button related to Employee is clicked.

Creating an account

This class allows the admin to create an employee.



Create Employee

Full Name

Username

Password

Register

```
public void addEmpInfo( ) {  
    try {  
        if (Database.connection == null) {  
            Database.connect();  
        }  
        Connection connection = Database.connection;  
        String Query = "Insert into empacc (empUserName, empPassword, empName) values (?,?,?)";  
        PreparedStatement stm = connection.prepareStatement(Query);  
        char[] passwordChars = password.getPassword();  
  
        // Convert char[] to String  
        String passwordString = new String(passwordChars);  
  
        // Store the string in your object  
        stm.setString(1, username.getText());  
        stm.setString(2, passwordString);  
        stm.setString(3, name.getText());  
  
        stm.executeUpdate();  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

addEmpInfo(), is designed to insert employee information into the "empacc" table of a database. It establishes a database connection, prepares an SQL query using a PreparedStatement to prevent SQL injection, and then sets the parameters with the values obtained from the corresponding user interface components (username, password, name). The method converts the password from a char array to a String before setting it in the PreparedStatement. Finally, the executeUpdate() method is called to execute the SQL insert query, adding the employee information to the database. We have used this in customer registration.

Validating the form

```
private boolean validateRegistration() {
    String enteredUserName = username.getText();
    char[] enteredPasswordChars = password.getPassword();
    String enteredPassword = new String(enteredPasswordChars);
    String enteredName = name.getText();

    if (enteredUserName.isEmpty() || enteredPassword.isEmpty() || enteredName.isEmpty()) {
        System.out.println("Please fill in all fields");
        return false;
    }

    return true;
}
```

validateRegistration(), aims to validate the user registration input by checking if the entered username, password, and name are not empty. It retrieves the text from corresponding user interface components (username, password, name), converts the password from a char array to a String, and then checks if any of these fields are empty. If any field is empty, a message is printed to the console indicating that all fields need to be filled, and the method returns false. Otherwise, it returns true, signifying that the registration input is valid. We have used this in customer registration.

Getting/Populating a table



Customer

Account ID	Username	Name
3	12	jack
6	JH	James Harden
7	CP	Chris Paul

Employee

Account ID	Username	Name
1	wewe	jamie
2	JC2002	james cooper

Username

12

Delete

Username

wewe

Delete

```

public void populateCustTable() {
    try {
        if (Database.connection == null) {
            Database.connect();
        }
        Connection connection = Database.connection;
        String query = "SELECT custID as 'Account ID', custUserName as 'Username', custName as 'Name' FROM shadowbank.custacc;";
        PreparedStatement stm = connection.prepareStatement(query);
        ResultSet result = stm.executeQuery();
        custTable.setModel(DbUtils.resultSetToTableModel(result));
    } catch (Exception e) {System.out.println(e);
    }
}

```

populateCustTable(), is designed to populate a table in a graphical user interface with customer information retrieved from the "custacc" table in a database. The method first checks if a database connection is established; if not, it connects to the database using the Database.connect() method. It then prepares an SQL query to select specific columns (custID, custUserName, custName) from the "custacc" table and executes the query. The ResultSet obtained from the query is converted to a TableModel using the DbUtils.resultSetToTableModel() method, and this model is set as the model for the custTable, which presumably represents a Swing JTable in the user interface. We have used this in different classes and changed query according to requirement.

Deleting an account

```

public void deleteEmpAcc() {
    try {
        if (Database.connection == null) {
            Database.connect();
        }
        Connection connection = Database.connection;
        String selectedEmpUserName = (String) empCB.getSelectedItem();
        String deleteQuery = "DELETE FROM shadowbank.empacc WHERE empUserName = ?";
        try (PreparedStatement deleteStm = connection.prepareStatement(deleteQuery)) {
            deleteStm.setString(1, selectedEmpUserName);
            int rowsAffected = deleteStm.executeUpdate();

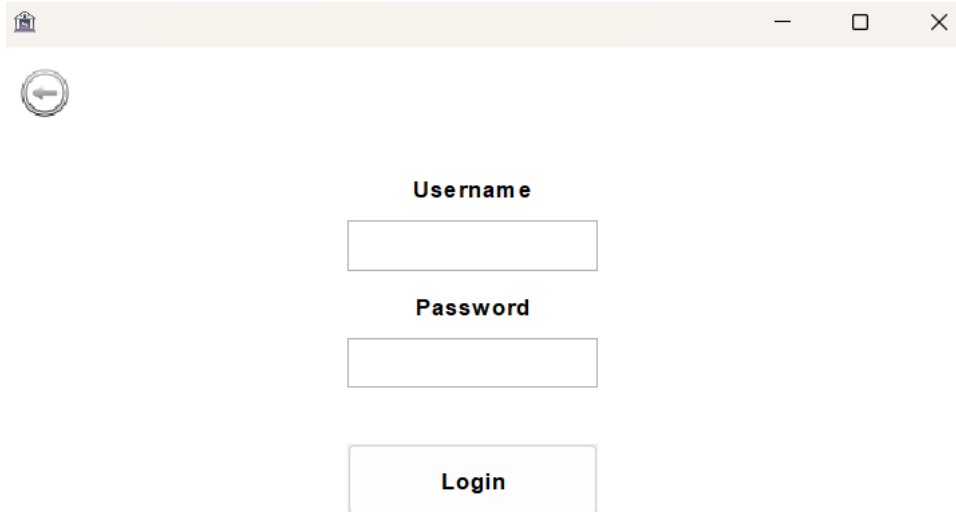
            if (rowsAffected > 0) {
                System.out.println("Account deleted successfully.");
                populateEmpTable();
            } else {
                System.out.println("Account with specified accID not found.");
            }
        } catch (Exception e) {
            e.printStackTrace();
            System.out.println("Error sending message: " + e.getMessage());
        }
    } catch (Exception e) {System.out.println(e);
    }
}

```

deleteEmpAcc(), is designed to delete an employee account from the "empacc" table in the database. The method first checks if a database connection is established; if not, it connects to the database using the Database.connect() method. It retrieves the selected employee username from a JComboBox (empCB), constructs an SQL DELETE query to remove the corresponding record from the "empacc" table, and executes the query using a PreparedStatement. The method then checks the number of rows affected by the deletion. If at least one row is affected, it prints a success message to the console, calls the populateEmpTable() method (presumably to update the

displayed table), and if no rows are affected, it prints a message indicating that the account was not found. We have used it in few different classes when deleting required. (query is change according to requirement)

Login



Username

Password

Login

```
public boolean confirmAdmininfo(String enteredUserName, String enteredPassword) {
    try {
        if (Database.connection == null) {
            Database.connect();
        }
        Connection connection = Database.connection;
        String query = "SELECT adminUserName, adminPassword FROM adminacc WHERE adminUserName = ?";
        PreparedStatement stm = connection.prepareStatement(query);
        stm.setString(1, enteredUserName);
        ResultSet resultSet = stm.executeQuery();
        if (resultSet.next()) {
            String storedPassword = resultSet.getString("adminPassword");
            if (enteredPassword.equals(storedPassword)) {
                return true;
            }
        }
        return false;
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}
```

confirmAdmininfo(), is designed to confirm the validity of administrator login credentials by querying the "adminacc" table in the database. The method first checks if a database connection is established; if not, it connects to the database using the Database.connect() method. It prepares an SQL SELECT query to retrieve the stored username and password for the specified administrator username. The method sets the entered username as a parameter in the PreparedStatement, executes the query, and retrieves the result set. If the result set has a next

record, indicating that the specified username exists, it compares the entered password with the stored password. If they match, the method returns true, indicating successful confirmation of administrator credentials; otherwise, it returns false. We have used the same for customer and employee login.

Requesting a bank Account

```
public void reqBA() throws SQLException {
    if (Database.connection == null) {
        Database.connect();
    }
    Connection connection = Database.connection;
    userModel UM = userModel.getInstance();
    String custUserName = UM.getUserName();
    String Query = "INSERT INTO shadowbank.barequest (custID, amount) SELECT custID, ? FROM shadowbank.custacc WHERE custUserName = ?";
    PreparedStatement stm = connection.prepareStatement(Query);
    stm.setInt(1, Integer.parseInt(amount.getText()));
    stm.setString(2, custUserName);
    stm.executeUpdate();
}
```

reqBA(), is designed to submit a bank account request by inserting a new record into the "barequest" table in the database. The method first checks if a database connection is established; if not, it connects to the database using the Database.connect() method. It obtains the current user's username through a singleton instance of the userModel class (UM). The method then prepares an SQL INSERT query to add a new record to the "barequest" table, selecting the customer ID and specified amount from the "custacc" table based on the provided username. The method sets the amount parameter and customer username as parameters in the PreparedStatement and executes the update. The insertion is successful, creating a bank account request for the specified customer. The method uses the amount field, presumably from a user interface component, to set the amount for the bank account request. The Database.connection and PreparedStatement resources are not explicitly closed in this method. It's recommended to use try-with-resources or ensure proper resource management to avoid potential resource leaks.

Accepting a bank account

```

private void handleApproval(boolean approve) {
    String reqIDString = request.getText();
    if (!reqIDString.isEmpty()) {
        try {
            int reqID = Integer.parseInt(reqIDString);
            if (approve) {
                // Update the approval status to true
                updateApprovalStatus(reqID, true);

                // If approved, insert into bankacc
                insertIntoBankAcc(reqID);
            } else {
                // Update the approval status to false
                updateApprovalStatus(reqID, false);
            }

            // Refresh the table after the update
            populateTable();
        } catch (NumberFormatException ex) {
            System.out.println(ex);
        }
    } else {
        System.out.println("Error");
    }
}

private void updateApprovalStatus(int reqID, boolean approve) {
    try {
        if (Database.connection == null) {
            Database.connect();
        }
        Connection connection = Database.connection;
        String updateQuery = "UPDATE shadowbank.barequest SET approval = ? WHERE requestID = ?";
        PreparedStatement updateStm = connection.prepareStatement(updateQuery);
        updateStm.setBoolean(1, approve);
        updateStm.setInt(2, reqID);
        updateStm.executeUpdate();
    } catch (Exception e) {
        System.out.print(e);
    }
}

private void insertIntoBankAcc(int reqID) {
    try {
        if (Database.connection == null) {
            Database.connect();
        }
        Connection connection = Database.connection;
        String query = "INSERT INTO shadowbank.bankacc (custID, amount) SELECT custID, amount FROM shadowbank.barequest WHERE requestID = ?";
        PreparedStatement stm = connection.prepareStatement(query);
        stm.setInt(1, reqID);
        stm.executeUpdate();
    } catch (Exception e) {
        System.out.print(e);
    }
}

```

The handleApproval method orchestrates the process by first extracting the request ID from a text field, parsing it into an integer, and then triggering either approval or rejection based on a boolean parameter. For approvals, it updates the approval status in the "barequest" table using the updateApprovalStatus method and inserts the corresponding request into the "bankacc" table through the insertIntoBankAcc method. The table displaying requests is subsequently refreshed. Both update operations safeguard against potential exceptions during database interactions. The updateApprovalStatus method modifies the approval status in the "barequest" table, while the insertIntoBankAcc method inserts the approved request into the "bankacc" table. This modular

approach enhances code readability and maintainability for managing the approval workflow in the banking application.

We have mostly used these different methods and changed them in other classes depending according to the requirement of the class.

Conclusion

In conclusion, the developed Java application for the banking system, referred to as the Shadow Bank, exhibits notable functionalities and features for managing bank accounts, employee profiles, and customer requests. However, certain limitations and challenges were encountered during the development process.

Limitations:

The application has a set of limitations, including challenges with encryption and code file management. Encryption, although a crucial aspect of security, might pose challenges in implementation, and code file management could benefit from optimization to enhance the overall maintainability of the system.

Challenges:

The development process faced challenges, particularly in ensuring the queries function as intended and effectively saving user data. Addressing these challenges required meticulous attention to detail and a thorough understanding of database interactions.

Future Upgrades:

For future upgrades, addressing the current limitations should be a priority, focusing on enhancing encryption methods and improving code file management for a more streamlined development environment. Additionally, a crucial aspect of future development lies in refining the user interface to provide a more intuitive and visually appealing experience for users.